

API Design Document

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 **Status:** Final for Implementation **Based On:** System Architecture v1.0, Database Design v1.0, Folder Structure v1.1

1. Purpose

This document specifies the complete REST API and WebSocket contract for SentinelCut's backend (FastAPI), implementing the routes scaffolded under `backend/app/api/v1/routes/`. It defines every endpoint's method, path, authentication requirement, request/response schema, and status codes, so frontend and backend development can proceed in parallel against a fixed contract.

2. API Conventions

2.1 Base URL & Versioning

```
Production: https://api.sentinelcut.app/api/v1
Local Dev:  http://localhost:8000/api/v1
```

All endpoints are versioned under `/api/v1`. Breaking changes will be introduced under `/api/v2` rather than mutating v1 in place.

2.2 Authentication

- Authenticated endpoints require a `Authorization: Bearer <jwt>` header.
- Guest-mode endpoints accept either a Guest Session token (same header format) or none, as noted per-endpoint.
- Admin endpoints require a valid JWT with `role: "admin"` claim; enforced via the `rbac_deps.py` dependency (per Folder Structure doc).

2.3 Standard Response Envelope

Success:

```
json
{
  "success": true,
  "data": { },
  "meta": { }
}
```

Error:

json

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "file_size_bytes exceeds maximum allowed size",
    "details": { "max_allowed_mb": 500 }
  }
}
```

2.4 Standard HTTP Status Codes Used

Code	Meaning	Usage
200	OK	Successful GET/PATCH/PUT
201	Created	Successful POST creating a resource
202	Accepted	Async job accepted (processing queued)
204	No Content	Successful DELETE
400	Bad Request	Validation failure
401	Unauthorized	Missing/invalid/expired token
403	Forbidden	Valid token, insufficient role/ownership
404	Not Found	Resource doesn't exist or not owned by requester
409	Conflict	Duplicate resource (e.g., preset name)
413	Payload Too Large	File exceeds max upload size
415	Unsupported Media Type	Invalid file format
422	Unprocessable Entity	Semantically invalid request (FastAPI default validation)
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unhandled server error
503	Service Unavailable	Processing queue/worker unavailable

2.5 Pagination

List endpoints accept `?page=1&page_size=20` and return:

json

```
"meta": {  
  "page": 1,  
  "page_size": 20,  
  "total_items": 143,  
  "total_pages": 8  
}
```

2.6 Rate Limiting

Per SRS-NFR-SEC-06, auth and upload endpoints are rate-limited (e.g., 10 requests/minute per IP for `/auth/*`, 20 requests/hour for `/files/upload` on guest sessions). Exceeding limits returns `429` with a `Retry-After` header.

3. Auth Routes (`auth_routes.py`)

3.1 `POST /auth/signup`

Auth: None **Request:**

json

```
{ "email": "user@example.com", "password": "SecurePass1!", "display_name": "Rohan" }
```

Response `201`:

json

```
{ "success": true, "data": { "user_id": "uuid", "email": "user@example.com", "acc
```

Errors: `400` (weak password), `409` (email already registered)

3.2 `POST /auth/login`

Auth: None **Request:** `{ "email": "...", "password": "..." }` **Response** `200`: `{ "access_token": "jwt", "refresh_token": "jwt", "user": {...} }` **Errors:** `401` (invalid credentials)

3.3 `POST /auth/google`

Auth: None **Request:** `{ "id_token": "google-issued-token" }` **Response** `200`: Same shape as login. Creates a new `users` record on first login (`auth_provider = 'google'`). **Errors:** `401` (invalid Google token)

3.4 POST /auth/forgot-password

Auth: None **Request:** { "email": "user@example.com" } **Response** (200): { "message": "If an account exists, a reset link has been sent." } (always 200 to avoid email enumeration)

3.5 POST /auth/reset-password

Auth: None (uses reset token from emailed link) **Request:** { "reset_token": "...", "new_password": "..." } **Response** (200): { "message": "Password updated successfully." } **Errors:** (400) (expired/invalid token)

3.6 POST /auth/guest-session

Auth: None **Response** (201): { "guest_token": "jwt", "expires_at": "2026-08-04T10:00:00Z" }

3.7 POST /auth/refresh

Auth: Refresh token in body **Request:** { "refresh_token": "jwt" } **Response** (200): { "access_token": "jwt" } **Errors:** (401) (expired/invalid refresh token)

3.8 POST /auth/logout

Auth: Required **Response** (204)

4. File Routes (file_routes.py)

4.1 POST /files/upload

Auth: Required (registered user or guest token) **Request:** multipart/form-data — field file (binary), original_filename (string) **Response** (201):

```
json
{
  "data": {
    "file_id": "uuid",
    "original_filename": "podcast_ep12.mp4",
    "file_type": "video",
    "file_size_bytes": 52428800,
    "status": "UPLOADED"
  }
}
```

Errors: (413) (too large), (415) (unsupported format), (401)

4.2 GET /files/{file_id}

Auth: Required (must own the file, or admin) **Response** (200): Full file record including current status, duration_seconds, detected_language. **Errors:** (404) (not found/not

owned)

4.3 GET /files

Auth: Required **Query params:** (?status=&search=&page=&page_size=&sort=-created_at

Response (200): Paginated list of the user's files (implements FR-DASH-01/02).

4.4 DELETE /files/{file_id}

Auth: Required (owner or admin) **Response** (204) (soft delete — sets deleted_at)

Errors: (404)

4.5 POST /files/{file_id}/process

Auth: Required **Request:**

```
json

{
  "detection_mode": "combined",
  "custom_words": ["badword1", "badword2"],
  "preset_id": "uuid|null",
  "replacement_mode": "beep"
}
```

Response (202): { "job_id": "uuid", "status": "QUEUED" } — enqueues the initial_processing job. **Errors:** (400) (invalid detection_mode combination), (404)

4.6 POST /files/{file_id}/regenerate

Auth: Required **Response** (202): { "job_id": "uuid", "status": "QUEUED" } — enqueues a regeneration job using current detections state. **Errors:** (404), (409) (file not in a regenerable status)

5. Detection Routes (detection_routes.py)

5.1 GET /files/{file_id}/detections

Auth: Required (owner) **Query params:** (?min_confidence=0.5) **Response** (200):

```
json
```

```
{
  "data": [
    {
      "id": "uuid",
      "detected_text": "bakchodi",
      "language": "hindi",
      "start_time": 12.450,
      "end_time": 13.100,
      "confidence_score": 0.94,
      "detection_source": "dictionary",
      "replacement_mode": "beep",
      "is_user_added": false,
      "is_dismissed": false
    }
  ]
}
```

5.2 POST /files/{file_id}/detections

Auth: Required (owner) — manually add a detection (FR-TLE-04) **Request:** `{ "detected_text": "...", "start_time": 45.2, "end_time": 45.8, "replacement_mode": "mute" }` **Response** (201): Created detection object with `is_user_added: true`. **Errors:** (400) (end_time <= start_time)

5.3 PATCH /detections/{detection_id}

Auth: Required (owner of parent file) **Request (any subset):** `{ "is_dismissed": true }` or `{ "replacement_mode": "custom_sound" }` or `{ "start_time": 45.1, "end_time": 45.9 }` **Response** (200): Updated detection object. **Errors:** (404)

5.4 PATCH /files/{file_id}/detections/batch

Auth: Required — bulk edit endpoint used by the Timeline Editor "save all changes" action **Request:**

json

```
{ "updates": [ { "id": "uuid", "is_dismissed": true }, { "id": "uuid", "replaceme
```

Response (200): `{ "updated_count": 2 }`

5.5 DELETE /detections/{detection_id}

Auth: Required (owner) **Response** (204)

6. Timeline Routes (`timeline_routes.py`)

6.1 `GET /files/{file_id}/transcript`

Auth: Required (owner) **Response** `200`:

```
json
{
  "data": {
    "full_text": "This movie is fucking amazing...",
    "segments": [
      { "text": "This movie is", "start": 0.0, "end": 1.2 },
      { "text": "fucking", "start": 1.2, "end": 1.6, "detection_id": "uuid" },
      { "text": "amazing", "start": 1.6, "end": 2.1 }
    ]
  }
}
```

6.2 `POST /files/{file_id}/editor/autosave`

Auth: Required — implements FR-TLE-08 (periodic autosave of draft edits) **Request:** `{ "draft_state": { } }` (opaque JSON blob of in-progress editor state) **Response** `200`: `{ "saved_at": "2026-08-03T10:15:00Z" }`

6.3 `GET /files/{file_id}/editor/autosave`

Auth: Required — recovers a draft after a crash/disconnect **Response** `200`: `{ "draft_state": { }, "saved_at": "..." }` or `404` if no draft exists

7. Export Routes (`export_routes.py`)

7.1 `GET /files/{file_id}/exports`

Auth: Required (owner) **Response** `200`: List of available export artifacts (`video`, `audio`, `srt`, `txt`, `json`) with generation status.

7.2 `GET /files/{file_id}/exports/{export_type}/download`

Auth: Required (owner) **Path param** `export_type`: one of `video | audio | srt | txt | json` **Response** `200`: `{ "download_url": "https://signed-url...", "expires_at": "..." }` — a short-lived signed Supabase Storage URL (per Architecture doc Section 7.1), not a proxied file stream. **Errors:** `404` (export not yet generated), `409` (file processing not complete)

8. Dashboard Routes (`dashboard_routes.py`)

8.1 `GET /dashboard/summary`

Auth: Required **Response** `200`:

```
json
{
  "data": {
    "recent_files": [ /* last 5 files */ ],
    "credits_remaining": 42,
    "storage_used_bytes": 1073741824,
    "storage_limit_bytes": 5368709120,
    "active_jobs": [ { "job_id": "uuid", "file_id": "uuid", "status": "PROCESSING" } ]
  }
}
```

9. Preset Routes (`preset_routes.py`)

9.1 `GET /presets`

Auth: Required — list the user's saved custom word-list presets.

9.2 `POST /presets`

Auth: Required **Request:** `{ "name": "My Podcast Filter", "word_list": ["word1", "word2"] }` **Response** `201` **Errors:** `409` (duplicate name for this user)

9.3 `PATCH /presets/{preset_id}`

Auth: Required (owner) **Request:** `{ "word_list": [...] }`

9.4 `DELETE /presets/{preset_id}`

Auth: Required (owner) **Response** `204`

10. Admin Routes (`admin_routes.py`)

All endpoints require `role: admin`; all mutating actions write to `admin_logs` (per Database Design Section 4.12).

10.1 `GET /admin/users`

Paginated list with filters `?status=active|disabled&search=`.

10.2 `PATCH /admin/users/{user_id}`

Request: `{ "status": "disabled" }` **Response** `200`

10.3 GET /admin/dictionary

Paginated list of (profanity_dictionary) entries, filterable by (? language=hindi&is_active=true).

10.4 POST /admin/dictionary

Request: { "term": "example", "language": "hindi", "severity": "high" } **Response** (201) **Errors:** (409) (duplicate normalized_term + language)

10.5 PATCH /admin/dictionary/{entry_id}

Request: { "is_active": false } or { "severity": "low" }

10.6 DELETE /admin/dictionary/{entry_id}

Response (204)

10.7 GET /admin/files

Paginated list of all files across all users, with filters (?status=&user_id=).

10.8 GET /admin/analytics

Response (200):

```
json
{
  "data": {
    "uploads_per_day": [ { "date": "2026-08-01", "count": 34 } ],
    "avg_processing_time_seconds": 87.4,
    "detection_stats": { "total_detections": 15234, "by_language": { "hindi": 6234 } }
  }
}
```

10.9 GET /admin/logs

Paginated (admin_logs) audit trail, filterable by (?admin_id=&target_entity=).

10.10 GET /admin/feedback

Paginated (feedback) list, filterable by (?status=open&category=bug).

10.11 PATCH /admin/feedback/{feedback_id}

Request: { "status": "resolved" }

11. Feedback Routes (`feedback_routes.py`)

11.1 `POST /feedback`

Auth: Optional (works for guest and registered users) **Request:** `{ "category": "detection_accuracy", "message": "Missed a Hinglish slang word." }` **Response** `201`

12. WebSocket Contract (`websocket_routes.py`)

12.1 Connection

```
WSS /ws/jobs/{job_id}?token=<jwt>
```

Authenticated via query-param JWT (validated on connection handshake). Rejects with close code `4401` if the token is invalid or the job doesn't belong to the requester.

12.2 Server → Client Messages

Status update:

json

```
{ "type": "status_update", "job_id": "uuid", "status": "PROCESSING", "stage": "asr" }
```

Completion:

json

```
{ "type": "job_completed", "job_id": "uuid", "file_id": "uuid", "status": "READY" }
```

Failure:

json

```
{ "type": "job_failed", "job_id": "uuid", "error_message": "ASR engine timeout", }
```

12.3 Client → Server Messages

json

```
{ "type": "ping" }
```

Server responds `{ "type": "pong" }` — used for connection keep-alive; no business logic accepted over the socket (all mutations go through REST, per the Architecture doc's separation of the WebSocket as a read-only status relay).

12.4 Fallback

If a client cannot establish a WebSocket connection (e.g., restrictive network), the frontend falls back to polling `GET /files/{file_id}` every 5 seconds (satisfies SRS-NFR-PERF-03's 5-second status latency requirement either way).

13. Error Code Reference

Code	HTTP Status	Meaning
VALIDATION_ERROR	400/422	Request body failed schema validation
AUTH_INVALID_CREDENTIALS	401	Login failed
AUTH_TOKEN_EXPIRED	401	JWT expired, refresh required
FORBIDDEN	403	Authenticated but not authorized for this resource
NOT_FOUND	404	Resource doesn't exist or isn't owned by requester
DUPLICATE_RESOURCE	409	e.g., duplicate preset name, duplicate dictionary term
FILE_TOO_LARGE	413	Exceeds configured max upload size
UNSUPPORTED_FORMAT	415	File format not in the supported list
RATE_LIMITED	429	Too many requests
PROCESSING_ERROR	500	AI pipeline failure after retries exhausted
QUEUE_UNAVAILABLE	503	Redis/worker pool unreachable

14. Endpoint Summary Table

Method	Path	Auth	Purpose
POST	/auth/signup	None	Register
POST	/auth/login	None	Login
POST	/auth/google	None	Google OAuth login
POST	/auth/forgot-password	None	Request reset link
POST	/auth/reset-password	None	Complete reset
POST	/auth/guest-session	None	Start guest session
POST	/auth/refresh	Refresh token	Refresh access token
POST	/auth/logout	Required	Logout
POST	/files/upload	Required	Upload media
GET	/files/{id}	Required	Get file detail
GET	/files	Required	List files
DELETE	/files/{id}	Required	Soft-delete file
POST	/files/{id}/process	Required	Start AI processing
POST	/files/{id}/regenerate	Required	Regenerate after edits
GET	/files/{id}/detections	Required	List detections
POST	/files/{id}/detections	Required	Add manual detection

Method	Path	Auth	Purpose
PATCH	/detections/{id}	Required	Edit a detection
PATCH	/files/{id}/detections/batch	Required	Bulk-edit detections
DELETE	/detections/{id}	Required	Remove a detection
GET	/files/{id}/transcript	Required	Get aligned transcript
POST	/files/{id}/editor/autosave	Required	Autosave draft edits
GET	/files/{id}/editor/autosave	Required	Recover draft
GET	/files/{id}/exports	Required	List export artifacts
GET	/files/{id}/exports/{type}/download	Required	Get signed download URL
GET	/dashboard/summary	Required	Dashboard data
GET/POST/PATCH/DELETE	/presets	Required	Manage presets
GET/PATCH	/admin/users	Admin	Manage users
GET/POST/PATCH/DELETE	/admin/dictionary	Admin	Manage profanity dictionary
GET	/admin/files	Admin	View all files
GET	/admin/analytics	Admin	Platform analytics
GET	/admin/logs	Admin	Audit trail

Method	Path	Auth	Purpose
GET/PATCH	/admin/feedback	Admin	Manage feedback
POST	/feedback	Optional	Submit feedback
WSS	/ws/jobs/{job_id}	Required	Real-time job status

15. Next Document

Document 9: AI Pipeline — detailed technical design of the `ai-pipeline/` module internals (ASR invocation, normalization rules, hybrid detection scoring logic, audio editing algorithms) that the `POST /files/{id}/process` and `/regenerate` endpoints trigger via the job queue.

End of API Design Document